

Vue, React, Next.js를 아우르는 확장 가능한 프론트엔드 보일러플레이트 설계

런타임 API 안정성, UI와 로직 분리, Storybook 기반 UI 문서화, 프레임워크별 상태 소유권을 중심으로 정리한 프론트엔드 아키텍처 프로젝트입니다.

역할
프론트엔드 아키텍처 / UI 시스템

유형
개인 아키텍처 프로젝트

범위
Vue, React, Next.js 보일러플레이트

초점
타입 안정성, RSC 경계, 상태 전략

Vue 보일러플레이트

Pinia, Vue Router, Atomic UI, Storybook, DTO 검증

React 보일러플레이트

TanStack Query, Zustand, HOC, MSW, View/Container 구조

Next.js 보일러플레이트

Server Components, Server Actions, 쿠키 세션, hydration-safe 날짜 처리

서버 우선 상태 소유권

요약

프로젝트 개요

목표는 단일 화면을 만드는 것이 아니라, 반복되는 초기 설정 비용을 줄이고 성장하는 프론트엔드 애플리케이션에 예측 가능한 개발 규칙을 세우는 것이었습니다.

과제 01

런타임 API 안정성

TypeScript만으로는 실제 API payload를 검증할 수 없어 API 경계에 DTO 검증을 추가했습니다.

과제 02

UI와 로직 분리

React는 View/Container/HOC, Vue는 composable/store, Next.js는 server/client 소유권 분리를 적용했습니다.

과제 03

Hydration과 상태 소유권

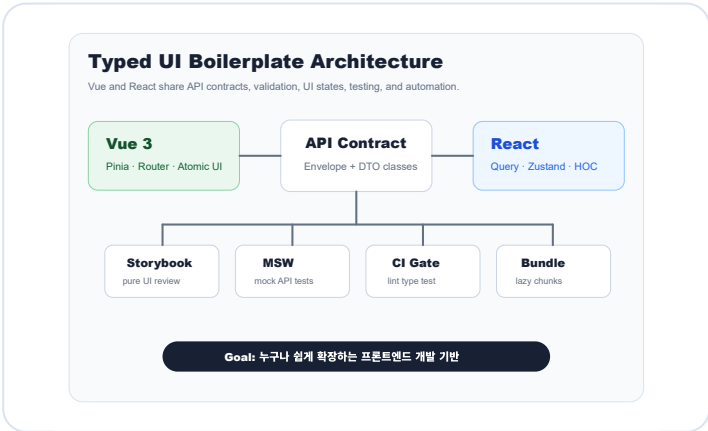
Next.js 상태를 server data, client cache, UI state, URL state, mutation 기준으로 분리했습니다.

아키텍처

공통 설계 원칙

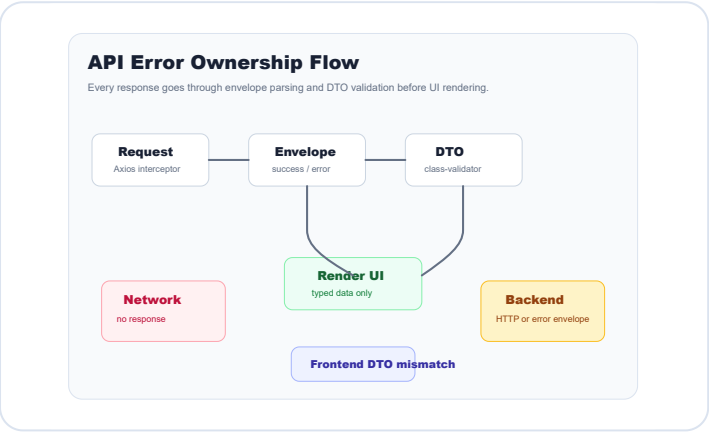
각 보일러플레이트는 프레임워크 런타임의 차이를 존중하면서도 같은 품질 기준을 따릅니다. API 계약, 상태 소유권, UI 상태, 테스트, 문서화를 모두 아키텍처의 일부로 다뤘습니다.

- UI 렌더링 전에 API envelope와 DTO 클래스를 검증
- 실패 원인을 프론트 계약, 백엔드, 네트워크, 인증 문제로 분류
- 로딩, 빈 상태, 오류 상태를 재사용 가능한 UI로 유지
- lazy route boundary와 scoped import를 우선 적용
- 순수 UI는 Storybook으로 문서화하고 계약은 테스트로 검증



프레임워크별 전략

Vue 3		
상태	Pinia	
라우팅	Vue Router lazy route	
UI	Atomic components + Storybook	
목적	템플릿 기반의 빠른 화면 구성	
React		
서버 상태	TanStack Query	
클라이언트 상태	Zustand	
구조	View, Container, hook, HOC	
목적	테스트 가능한 UI/로직 분리	
Next.js		
초기 데이터	Server Component	
클라이언트 갱신	TanStack Query	
UI 상태	Zustand	
변경 작업	Server Action	



API 안정성

타입 기반 API와 오류 소유권

API 응답은 envelope 파싱과 DTO 검증을 거칩니다. DTO 불일치는 프론트엔드 계약 문제로 보고, HTTP 실패는 백엔드 또는 네트워크 문제로 분류했습니다.

```
{
  "success": true,
  "data": {
    "id": "u-1",
    "email": "demo@example.com"
  }
}
```

운영형 프론트엔드 콘솔

Next.js 보일러플레이트를 시니어 프론트엔드 역할에 맞는 증거 화면으로 확장했습니다. B2B 운영 콘솔 안에 런타임 DTO 검증, i18n, 클라이언트 실시간 업데이트, 릴리즈 모니터링, 디자인 시스템 재사용 경험을 함께 담았습니다.

● Global Ops Console

KO / EN

파트너 전환율

34.8%

good

API 지연시간 p95

184ms

watch

프론트 오류율

0.18%

good

LCP

1.9s

good

실시간 모니터

WebSocket 연결이 재시도 후 정상 복구되었습니다.

릴리즈 파이프라인

프로덕션 배포가 CI 상태와 함께 검증되었습니다.

서버 계약 검증

UI 렌더링 전에 class-validator DTO로 API envelope를 검증합니다.

OpsConsole.dto.ts

i18n

타입이 잡힌 dictionary와 클라이언트 상태로 KO/EN 전환을 처리합니다.

opsDictionary

실시간 UI

서버에서 검증한 초기 데이터 위에 클라이언트 실시간 이벤트를 피드를 엮었습니다.

LiveIncidentFeed

운영 관점

Core Web Vitals, 오류율, 지연시간, 릴리즈 상태를 운영 지표로 노출합니다.

/ops-console

WOD Planner

CrossFit WOD와 HYROX 운동을 날짜별로 저장하고, 출석, 개인 기록, 통증 상태, 부위별 피로도를 함께 관리하는 개인 운동 플래닝 앱입니다. 단순 기록장을 넘어 다음 운동의 보조 운동 추천까지 이어지도록 설계했습니다.

WOD 플래너 MVP2026-06-04

오늘 WOD

기록 경쟁 WOD

400m 러닝 · 월볼 20회 · 버피 15회

부위별 피로

대퇴사두

어깨

허리

추천

상체 보조 운동

하체 피로가 높아
오버헤드와 코어 중
심 보조 운동을 우
선 추천합니다.

WOD 파
서

날짜, 섹션,
타임캡, 동작
을 텍스트에
서 추출하고
CrossFit/HYROX
타입을 구분
합니다.

wodParser.ts

피로도 계
산

오늘/내일
WOD, 최근
운동, 통증 플
래그를 합산
해 부위별 피
로 점수를 계
산합니다.

fatigueScorer.ts

운동 추천

하체, 후면사
슬, 어깨 피로
와 통증 상태
를 바탕으로
피해야 할 동
작과 추천 보
조 운동을 제
안합니다.

workoutRecommendation.ts

기록 관리

출석 체크,
WOD 결과,
세트별 기록,
PB, 동작 빈
도, 능력치 요
약을 브라우
저 로컬 상태
로 관리합니
다.

localStorage

AI는 초안 엔진으로 쓰고, 아키텍처 판단은 사람이 소유합니다

보일러플레이트 위에 AI 활용 워크플로우를 추가했습니다. 구현 초안은 AI에게 맡기되, 런타임 경계, API 계약, 상태 소유권, 성능, 접근성, 릴리즈 리스크 같은 시니어 프론트엔드 판단은 사람이 검토하고 결정하는 구조입니다.

1

프롬프트 계약

코드 생성 전에 목표, 범위, 제약, 검증 명령, 리뷰 관점을 먼저 정의합니다.

PROMPT_PLAYBOOK.md

2

AI 초안 생성

UI 초안, DTO/테스트 초안, 리팩토링 후보 탐색, E2E 시나리오 생성을 AI에게 맡깁니다.

AI_WORKFLOW.md

3

사람의 리뷰

Boundary 위치, 상태 소유권, API 계약, 접근성, 번들 영향을 사람이 검토합니다.

CODE_REVIEW_CHECKLIST.md

4

검증 게이트

자동화 검증이 통과해야만 AI가 만든 코드가 아키텍처에 맞는 것으로 인정합니다.

lint / typecheck / test / build / e2e

리팩토링 재현 사례

Before / After: AI가 만든 리팩토링을 사람이 통제하는 방식

의도적으로 복잡하게 만든 운영 테이블 컴포넌트를 사용해, AI 초안을 기반으로 경계가 명확하고 테스트 가능한 구조로 나누는 과정을 보여줍니다.

이전

LegacyOpsTable.client.tsx

- 검색 상태, 필터링, 라벨, 테이블 렌더링이 하나의 client 파일에 섞여 있음
- 비즈니스 라벨이 JSX 내부에 직접 박혀 있음
- 전체 컴포넌트를 렌더링하지 않으면 테스트하기 어려움
- 서버 데이터 구조로 옮기기 전에 client boundary가 불필요하게 큼

Client component

- 입력 상태
- 필터 로직
- 상태 라벨
- 테이블 마크업

이후

OpsTable.client.tsx + OpsTableView

- client 컴포넌트는 검색어 상태만 소유
- 필터링과 상태 라벨 로직을 pure function으로 분리
- View 계층은 서버에서 검증된 row 데이터를 받을 수 있음
- 브라우저 렌더링 없이 로직 회귀 테스트 가능

Client component

- 로컬 검색 상태

Pure logic

- filterOpsRows()
- getStatusLabel()

View

- DataTable

AI는 초안을 만들고, 사람은 제품 수준의 판단을 소유합니다

AI에게 맡기는 일	사람이 책임지는 일
컴포넌트 초안과 반복 UI variant 생성	Server/Client Component 경계와 hydration 리스크 판단
DTO, schema, 테스트 초안 작성	API 계약 형태와 하위 호환성 결정
리팩토링 후보 탐색과 기계적인 분리 작업	상태 소유권 결정: server, cache, URL, local, Zustand
Storybook 상태와 E2E 시나리오 초안 생성	접근성 기준, UX 수용 기준, 릴리즈 리스크 판단

검증 파이프라인

AI 결과물은 파이프라인을 통과하기 전까지 받아들이지 않습니다

```
npm run lint
npm run typecheck
npm run test
npm run build
npm run e2e
npm run check:automation
```

12
테스트 파일

25
unit / a11y
테스트

8
E2E 테스트

Pass
프로덕션 빌드

React, Vue, Next 보일러플레이트를 하나의 프론트엔드 플랫폼으로 정렬

세 개의 보일러플레이트를 단순 샘플 프로젝트가 아니라, 여러 앱과 공통 패키지가 같은 품질 기준을 공유하는 모노레포형 프론트엔드 플랫폼 사례로 재구성했습니다.

목표 모노레포 구조

```
frontend-platform
apps/
  next-web
  react-admin
  vue-console
packages/
  ui
  contracts
  config
  testing
  ai-workflow
```

apps

Next, React, Vue 앱은 각 런타임 특성을 유지하되 동일한 품질 게이트를 공유합니다.

next / react / vue

packages/ui

공통 UI, 상태 컴포넌트, Storybook 문서화를 패키지 단위로 관리하는 구조를 가정합니다.

design system

packages/contracts

DTO, schema, API envelope, mock scenario를 앱과 분리해 계약 변경 영향을 줄입니다.

DTO / Zod / MSW

packages/ai-workflow

AI 프롬프트, 리뷰 체크리스트, check:ai 자동화를 공통 품질 기준으로 묶습니다.

check:ai

프레임워크가 달라도 같은 검증 파이프라인을 통과해야 합니다

GitHub Actions와 npm script를 중심으로 품질 게이트를 구성했습니다. Jenkins를 쓰더라도 같은 npm script를 호출하면 동일한 검증 기준을 재사용할 수 있습니다.

01

install

lockfile 기반 의존성 설치

02

lint

금지 패턴과 컨벤션 검사

03

typecheck

strict TypeScript 검증

04

test

unit, DTO, a11y 테스트

05

check:ai

AI 문서, PR 템플릿, 리뷰 기준 누락 검사

06

build

프로덕션 번들 생성

07

bundle

번들 예산과 dependency 크기 검사

08

storybook

공통 UI 문서 빌드

```
GitHub Actions / Jenkins
→ npm run check:ci
→ npm run check:ai
→ npm run check:bundle
→ npm run build-storybook
```

핵심은 특정 CI 도구가 아니라, 품질 기준을 npm script로 고정해 GitHub Actions와 Jenkins 양쪽에서 같은 검증을 재사용할 수 있게 만드는 것입니다.

검증된 산출물

보일러플레이트에는 CI 명령어, Storybook, DTO 테스트, client/server 경계 문서, 보안 감사 확인 항목이 포함되어 있습니다.

CI 게이트

lint, typecheck, test, production build, Storybook build를 검증합니다.

check:ci

보안 점검

moderate 수준의 audit 검사가 통과되도록 기준을 둡니다.

npm audit

Storybook

순수 UI 컴포넌트를 독립적으로 문서화합니다.

Button, Card

MSW

mock API 계약으로 프론트엔드 테스트를 지원합니다.

handlers.ts

Next 경계

상태 소유권, RSC 규칙, hydration-safe 날짜 hook을 정리했습니다.

BOUNDARY_GUIDE

운영 콘솔

DTO 검증, i18n, live feed, Web Vitals, 릴리즈 상태를 포함합니다.

/ops-console

핵심 결정

런타임 DTO 검증

UI가 데이터를 받기 전에 런타임에서 검증해 API 계약 drift를 줄였습니다.

Next.js 상태 소유권

초기 데이터는 server-first로 유지하고, TanStack Query는 인터랙티브 갱신에만 사용했습니다.

styled-components 제외

CSS Modules, SCSS Modules, CSS variables로 Server Components와 hydration을 단순하게 유지했습니다.

부록

프로젝트 구조

폴더 경계는 이후 AI 보조 개발에서도 구조를 쉽게 추론할 수 있도록 설계했습니다.

```
react-boilerplate
src/components
src/features
src/hoc
src/hooks
src/core
src/test/msw
```

```
boilplate
src/components
src/stores
src/router
src/api
tests/performance
storybook
```

```
next-boilerplate
app
src/features/*/server
src/features/*/actions
src/components/client
src/hoc/server
src/stores
```

다음 단계

확장 계획

1

디자인 토큰

Figma token을 CSS variables와 프레임워크별 preset으로 연결합니다.

2

생성기

feature, DTO, story, test, route 파일을 일관된 규칙으로 생성합니다.

3

프로덕션 인증

server boundary를 유지하면서 demo session을 실제 인증 provider로 교체합니다.